Crop Disease Identification

1. Abstract

Agriculture is one of the most important sectors contributing to food security and economic development. However, plant diseases significantly affect crop production and quality. Early detection of plant diseases is essential for preventing large-scale crop losses. Traditional disease identification methods rely on manual inspection by experts, which can be time-consuming and expensive.

This project presents a machine learning–based approach for detecting plant diseases using leaf images. The system uses the PlantVillage dataset containing more than 54,000 images across 38 plant–disease classes. Three machine learning models—Support Vector Machine (SVM), Random Forest, and Convolutional Neural Network (CNN)—are developed and compared. The CNN model achieved the best performance with an accuracy of approximately 87%.

A user-friendly web application built with Streamlit allows users to upload leaf images and receive instant predictions about plant health or disease type. The system demonstrates the potential of artificial intelligence in agriculture for improving crop monitoring and disease management.

2. Introduction

Agriculture is a fundamental sector that supports human survival by providing food, raw materials, and employment opportunities. Plant diseases pose a major threat to agricultural productivity. Diseases caused by fungi, bacteria, viruses, and environmental conditions can damage crops and reduce yield.

Traditionally, farmers detect plant diseases through visual observation or consultation with agricultural experts. However, this approach has several limitations:

- It requires expert knowledge.
- It may not detect diseases at an early stage.
- It is time-consuming and expensive.

With the advancement of computer vision and machine learning technologies, automated plant disease detection systems can be developed. These systems analyze images of plant leaves and identify disease patterns.

In this project, we propose a machine learning system that detects plant diseases using leaf images. The system combines image processing techniques with machine learning algorithms to classify plant diseases accurately.

The final model is integrated into a web application so that users can easily upload leaf images and obtain predictions.

3. Objectives

The main objectives of this project are:

1. To collect and preprocess plant leaf image data.
2. To develop machine learning models for plant disease classification.
3. To compare the performance of different models such as SVM, Random Forest, and CNN.
4. To build an interactive web application for disease detection.
5. To demonstrate the use of artificial intelligence in agricultural applications.

4. Dataset Description

The dataset used in this project is the **PlantVillage dataset**, which is publicly available on Kaggle.

Dataset Features

- Total images: Approximately **54,000**
- Number of classes: **38 plant–disease categories**
- Image type: RGB leaf images
- Dataset format: Image folders organized by class labels

Each folder represents a specific plant and disease combination.

Examples of classes include:

- Apple Scab
- Apple Black Rot
- Tomato Early Blight
- Tomato Late Blight
- Corn Common Rust
- Potato Early Blight
- Grape Black Rot
- Healthy leaves for various plants

The dataset is widely used for plant disease research and machine learning experiments

5. System Architecture

The proposed system consists of several stages, starting from data preprocessing to model deployment.

Steps in the system

1. Data Collection
2. Data Cleaning
3. Data Validation
4. Data Augmentation
5. Feature Engineering
6. Train-Test Split
7. Feature Processing (PCA)
8. Model Training
9. Model Evaluation
10. Frontend Deployment

The trained model is integrated into a Streamlit web application where users can upload leaf images and obtain disease predictions.

6. Methodology

6.1 Data Preprocessing

Data preprocessing is essential to ensure that the dataset is clean and suitable for training machine learning models.

The following steps were performed:

- Removal of corrupted images
- Verification of image labels
- Image resizing
- Pixel value normalization
- Data augmentation

All images were resized to 32 × 32 pixels to reduce computational complexity.

Data augmentation techniques such as rotation, flipping, and zooming were used to increase the diversity of training images.

6.2 Feature Processing

For classical machine learning models such as SVM and Random Forest, feature extraction is necessary.

Principal Component Analysis (PCA) was used to reduce the dimensionality of the dataset while preserving important information.

Advantages of PCA include:

- Reduced computational cost
- Removal of redundant features
- Improved model performance

6.3 Machine Learning Models

Support Vector Machine (SVM)

SVM is a supervised learning algorithm used for classification tasks. It works by finding the optimal hyperplane that separates different classes in the dataset.

Advantages:

- Effective for high-dimensional data
- Good performance with small datasets

Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to make predictions.

Advantages:

- Handles large datasets
- Reduces overfitting
- Provides stable results

Convolutional Neural Network (CNN)

CNN is a deep learning model specifically designed for image recognition tasks.

The CNN architecture used in this project consists of:

- Convolution layers
- Batch normalization

- Max pooling layers
- Fully connected layers
- Dropout layer
- Softmax output layer

CNN automatically learns image features such as edges, textures, and patterns.

7. Model Training

The CNN model was trained using an image generator to efficiently handle large datasets without loading all images into memory.

Training Parameters

- Input size: 32 × 32 × 3
- Optimizer: Adam
- Loss Function: Categorical Crossentropy
- Epochs: 10
- Batch size: 32

The dataset was divided into training and testing sets to evaluate model performance.

8. Model Evaluation

The performance of the models was evaluated using classification accuracy.

Model	Accuracy
Support Vector Machine	~70%
Random Forest	~66%
Convolutional Neural Network	~87%

The CNN model achieved the highest accuracy and was selected for deployment.

9. Frontend Deployment

A web application was developed using **Streamlit**, a Python framework for building interactive data applications.

Features of the Application

- Upload plant leaf images
- Run disease detection
- Display predicted plant species
- Display disease name
- Show prediction confidence
- Display top five predicted classes

The trained CNN model is loaded into the application to perform real-time predictions.

10. Results and Discussion

The proposed system successfully identifies plant diseases using leaf images. The CNN model demonstrated strong performance compared to traditional machine learning models.

The Streamlit interface allows users to easily interact with the model. Users can upload images and receive predictions within seconds.

The system can help farmers and agricultural researchers identify plant diseases early and take preventive measures.

11. Applications

This system can be used in several agricultural applications:

- Smart farming systems
- Crop monitoring tools
- Agricultural research
- Mobile farming applications
- Precision agriculture

Such technologies can improve crop health monitoring and reduce agricultural losses.

12. Limitations

Despite its advantages, the system has some limitations:

- Model accuracy depends on image quality.
- Field images may differ from laboratory dataset images.
- The model cannot detect diseases not present in the training dataset.

Future improvements can address these challenges.

13. Future Work

Future enhancements for this project include:

- Training models with larger image sizes for better accuracy.
- Using advanced deep learning architectures such as ResNet or EfficientNet.
- Deploying the system as a mobile application.
- Providing disease treatment recommendations.
- Integrating real-time camera detection.

14. Conclusion

This project demonstrates the use of machine learning and deep learning techniques for plant disease detection. The CNN model achieved the best performance among the tested models.

The integration of the model into a Streamlit web application allows users to easily upload leaf images and obtain disease predictions.

This system can assist farmers in identifying plant diseases early, improving crop health management and agricultural productivity.

Coding

```
import os
import gc
import pickle
import numpy as np
import cv2
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (Conv2D, MaxPooling2D, Flatten,
                                     Dense, Dropout, BatchNormalization)
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print(" ✅ All libraries imported successfully.")


dataset_path = r"D:\VS Code\Python\plant disease idetification\plantvillage dataset\color"
removed = 0
for folder in os.listdir(dataset_path):
```

```
    folder_path = os.path.join(dataset_path, folder)
    if not os.path.isdir(folder_path):
        continue
    for fname in os.listdir(folder_path):
        fpath = os.path.join(folder_path, fname)
        if not fname.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp')):
            continue
        try:
            img = Image.open(fpath)
            img.verify()
        except Exception:
            print(f"Removing corrupted: {fpath}")
            os.remove(fpath)
            removed += 1

print(f"✅ Data Cleaning complete. Removed {removed} corrupted file(s).")
classes = sorted([
    f for f in os.listdir(dataset_path)
    if os.path.isdir(os.path.join(dataset_path, f))
])
print(f"Total classes found: {len(classes)}\n")

total_images = 0
empty_classes = []
for cls in classes:
    cls_path = os.path.join(dataset_path, cls)
    imgs = [f for f in os.listdir(cls_path)
            if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp'))]
    print(f"  {cls:<60} → {len(imgs):>5} images")
    total_images += len(imgs)
    if len(imgs) == 0:
```

```
        empty_classes.append(cls)

print(f"\nTotal images across all classes: {total_images}")
if empty_classes:
    print(f"⚠️  Empty classes: {empty_classes}")
else:
    print("✅ Data Validation complete. No empty classes found.")
# Global size and batch constants used in all subsequent steps
IMAGE_SIZE = 32      # 32×32 → streams from disk with minimal RAM
BATCH_SIZE = 32

# Training generator: rescale + augmentation
datagen_train = ImageDataGenerator(
    rescale         = 1./255,
    validation_split = 0.2,       # 80/20 split handled inside the generator
    rotation_range   = 20,
    width_shift_range= 0.1,
    height_shift_range=0.1,
    horizontal_flip  = True,
    zoom_range       = 0.1,
    fill_mode        = 'nearest'
)

# Validation generator: rescale only (no augmentation on val data)
datagen_val = ImageDataGenerator(
    rescale         = 1./255,
    validation_split = 0.2
)

print("✅ Augmentation generators configured.")
```

```
print("   Training  : rescale + rotation + shift + flip + zoom")
print("   Validation: rescale only")

# ── CNN generators (stream from disk — zero full-array RAM) ──────────────────────────

train_gen = datagen_train.flow_from_directory(
    dataset_path,
    target_size  = (IMAGE_SIZE, IMAGE_SIZE),
    batch_size   = BATCH_SIZE,
    class_mode   = 'categorical',
    subset       = 'training',
    shuffle      = True,
    seed         = 42
)

val_gen = datagen_val.flow_from_directory(
    dataset_path,
    target_size  = (IMAGE_SIZE, IMAGE_SIZE),
    batch_size   = BATCH_SIZE,
    class_mode   = 'categorical',
    subset       = 'validation',
    shuffle      = False,
    seed         = 42
)

num_classes = train_gen.num_classes
class_names = list(train_gen.class_indices.keys())

print(f"\n✅ Generators ready — images stream from disk, not RAM.")
print(f"   Training   batches : {len(train_gen)}  ({train_gen.n} images)")
print(f"   Validation batches : {len(val_gen)}  ({val_gen.n} images)")
```

```
print(f"   Classes          : {num_classes}")
# Fit encoder on class names from the generator
encoder = LabelEncoder()
encoder.fit(class_names)

print(f"✅ LabelEncoder fitted on {len(encoder.classes_)} classes.")
print(f"   First 5: {list(encoder.classes_)[:5]} ...")
print()
print("   Train/Val split is already handled by flow_from_directory")
print(f"   Training images  : {train_gen.n}")
print(f"   Validation images: {val_gen.n}")
SAMPLE_SIZE = 5000   # sufficient for SVM/RF, keeps RAM low

# Separate generator — no augmentation, no validation_split
datagen_flat = ImageDataGenerator(rescale=1./255)
flat_gen = datagen_flat.flow_from_directory(
    dataset_path,
    target_size = (IMAGE_SIZE, IMAGE_SIZE),
    batch_size  = BATCH_SIZE,
    class_mode  = 'sparse',    # integer labels
    shuffle     = True,
    seed        = 42
)

# Collect SAMPLE_SIZE images
sample_x, sample_y = [], []
for imgs, lbls in flat_gen:
    sample_x.append(imgs)
    sample_y.append(lbls)
    collected = sum(len(b) for b in sample_x)
```

```
    if collected >= SAMPLE_SIZE:
        break

sample_x = np.concatenate(sample_x, axis=0)[:SAMPLE_SIZE]   # (5000, 32, 32, 3)
sample_y = np.concatenate(sample_y, axis=0)[:SAMPLE_SIZE].astype(int)

# Flatten → PCA
X_flat = sample_x.reshape(len(sample_x), -1).astype(np.float32)
del sample_x; gc.collect()

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_flat, sample_y,
    test_size  = 0.2,
    random_state = 42,
    stratify   = sample_y
)
del X_flat; gc.collect()

pca = PCA(n_components=100, random_state=42)
X_train_pca = pca.fit_transform(X_train_s)
X_test_pca  = pca.transform(X_test_s)
del X_train_s, X_test_s; gc.collect()

print(f"✅ Feature Processing complete.")
print(f"   Sample drawn       : {SAMPLE_SIZE} images")
print(f"   PCA input          : {X_train_pca.shape[1]+pca.n_components_} → 100 features")
print(f"   Train PCA shape    : {X_train_pca.shape}")
print(f"   Test  PCA shape    : {X_test_pca.shape}")
print(f"   Variance explained : {pca.explained_variance_ratio_.sum()*100:.1f}%")
print("Training Random Forest ...")
```

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train_pca, y_train_s)
print("✅ Random Forest training complete.")
gc.collect()
tf.keras.backend.clear_session()

model = Sequential([
    # Block 1
    Conv2D(32, (3,3), activation='relu', padding='same',
           input_shape=(IMAGE_SIZE, IMAGE_SIZE, 3)),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    # Block 2
    Conv2D(64, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    # Block 3
    Conv2D(128, (3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),

    # Head
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.4),
    Dense(num_classes, activation='softmax')
])

model.compile(
```

```
    optimizer = 'adam',
    loss      = 'categorical_crossentropy',
    metrics   = ['accuracy']
)
model.summary()
# Generator-based fit — only one batch (32 images) in RAM at a time
history = model.fit(
    train_gen,
    epochs          = 10,
    validation_data = val_gen,
    verbose         = 1
)
print("\n✅ CNN training complete.")
# ── SVM ──────────────
```

```
    preds = model.predict(imgs, verbose=0)
    y_pred_cnn.extend(np.argmax(preds, axis=1))
    y_true_cnn.extend(np.argmax(lbls,  axis=1))
    if len(y_true_cnn) >= val_gen.n:
        break
y_pred_cnn = np.array(y_pred_cnn[:val_gen.n])
y_true_cnn = np.array(y_true_cnn[:val_gen.n])

print(f"SVM Accuracy          : {svm_accuracy*100:.2f}%  (on {SAMPLE_SIZE}-sample
subset)")
print(f"Random Forest Accuracy : {rf_accuracy*100:.2f}%  (on {SAMPLE_SIZE}-sample
subset)")
print(f"CNN Accuracy          : {cnn_accuracy*100:.2f}%  (on full validation set)")
print("\n✅ Model Evaluation complete.")
fig, axes = plt.subplots(1, 3, figsize=(22, 6))

cm_data_list = [
    (confusion_matrix(y_test_s, svm_pred),  "SVM Confusion Matrix",          "Blues"),
    (confusion_matrix(y_test_s, rf_pred),   "Random Forest Confusion Matrix", "Greens"),
    (confusion_matrix(y_true_cnn, y_pred_cnn), "CNN Confusion Matrix",       "Oranges"),
]

for ax, (cm_data, title, cmap) in zip(axes, cm_data_list):
    sns.heatmap(cm_data, annot=False, fmt='d', cmap=cmap, ax=ax,
                xticklabels=False, yticklabels=False)
    ax.set_title(title, fontsize=13, fontweight='bold')
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.suptitle("Confusion Matrices — All Models", fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
```

```
plt.show()
target_names = [c.replace('___', ' — ').replace('_', ' ') for c in class_names]
print(classification_report(y_true_cnn, y_pred_cnn, target_names=target_names))
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(history.history['accuracy'],     label='Train')
axes[0].plot(history.history['val_accuracy'], label='Validation')
axes[0].set_title('CNN Accuracy per Epoch')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Accuracy')
axes[0].legend(); axes[0].grid(True)

axes[1].plot(history.history['loss'],     label='Train')
axes[1].plot(history.history['val_loss'], label='Validation')
axes[1].set_title('CNN Loss per Epoch')
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Loss')
axes[1].legend(); axes[1].grid(True)

plt.suptitle("CNN Training Curves", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
model_names = ['SVM', 'Random Forest', 'CNN']
accuracies  = [svm_accuracy*100, rf_accuracy*100, cnn_accuracy*100]
colors      = ['#4C72B0', '#55A868', '#C44E52']

plt.figure(figsize=(8, 5))
bars = plt.bar(model_names, accuracies, color=colors, edgecolor='black', width=0.5)
for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
             f'{acc:.2f}%', ha='center', va='bottom', fontweight='bold')
```

```
plt.ylim(0, 110)
plt.title('Model Accuracy Comparison', fontsize=14, fontweight='bold')
plt.ylabel('Accuracy (%)')
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

print("\n📊 Final Summary:")
for name, acc in zip(model_names, accuracies):
    print(f"  {name:<20}: {acc:.2f}%")
best = model_names[accuracies.index(max(accuracies))]
print(f"\n🏆 Best Model : {best}")
# Save CNN
model.save('plant_disease_cnn.h5')

# Save SVM, RF, PCA, Encoder
with open('svm_model.pkl',  'wb') as f: pickle.dump(svm_model, f)
with open('rf_model.pkl',   'wb') as f: pickle.dump(rf_model,  f)
with open('pca_encoder.pkl', 'wb') as f:
    pickle.dump({'pca': pca, 'encoder': encoder}, f)

print("✅ Saved:")
print("   plant_disease_cnn.h5")
print("   svm_model.pkl | rf_model.pkl | pca_encoder.pkl")
def predict_plant_disease(image_path: str, model_type: str = 'cnn') -> dict:
    """
    Predict plant disease from a leaf image.

    Parameters
    ----------
```

```
    image_path : str        path to a .jpg / .png leaf image
    model_type : str        'cnn' | 'svm' | 'rf'

    Returns
    -------
    dict  {'predicted_class': str, 'confidence': str}  (confidence only for CNN)
    """
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Cannot read image: {image_path}")
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMAGE_SIZE, IMAGE_SIZE)).astype(np.float32) / 255.0

    if model_type == 'cnn':
        probs      = model.predict(img[np.newaxis, ...], verbose=0)[0]
        pred_idx   = int(np.argmax(probs))
        confidence = float(probs[pred_idx])
        label      = encoder.classes_[pred_idx]
        return {"predicted_class": label, "confidence": f"{confidence*100:.2f}%"}

    elif model_type in ('svm', 'rf'):
        flat     = img.flatten().reshape(1, -1)
        flat_pca = pca.transform(flat)
        clf      = svm_model if model_type == 'svm' else rf_model
        pred     = clf.predict(flat_pca)[0]
        return {"predicted_class": encoder.classes_[pred]}

    raise ValueError("model_type must be 'cnn', 'svm', or 'rf'")

print("✅ predict_plant_disease() ready.")
```

```
print("   predict_plant_disease('leaf.jpg', model_type='cnn')")
```